

Recurrent Neural Networks

Back-propagation Through Time and Bidirectional RNNs

Introduction to Deep Learning · Class 26

Outline

From Space to Time

The Recurrent Network

Back-propagation Through Time

The Cost of Time

Both Directions

Putting It Together

From Space to Time

Where we are coming from

Classes 23–25: convolution

the input is a **grid** of fixed size — an image

nearby pixels matter most

a cat is a cat wherever it sits

a kernel has few weights, an image many pixels

what it gave us

one kernel slides over every position

locality: each output sees a small window

equivariance: the same weights everywhere

parameter sharing across **space**

- All of it rests on the input being a fixed-size grid in which position is **where**
- Today's data is an **ordered list of variable length**, position is **when**, and what matters may be far back
- The instinct carries over — one set of weights, reused everywhere — but “everywhere” will mean every **time step**, plus one new wire

Classes 23–25; Goodfellow et al. (2016), §10 opening; Jurafsky & Martin (2026), Ch. 14 opening.

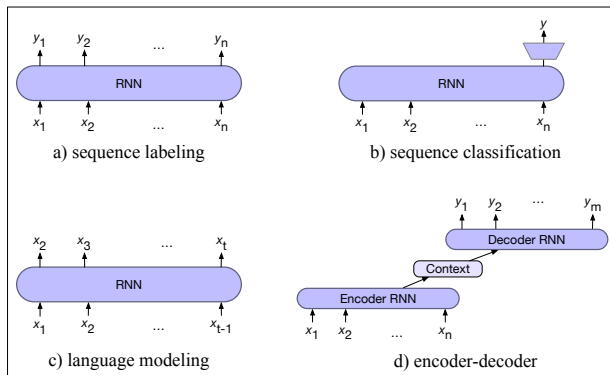
The data: sequences

- A **sequence** is an ordered list $\mathbf{x}_1, \dots, \mathbf{x}_\tau$ whose length τ varies: the words of a sentence, the frames of a recording, the readings of a sensor
- **Order matters**: “dog bites man” and “man bites dog” have the same words; a bag of features loses the point
- **Length varies**: a five-word and a fifty-word sentence are both sentences; a fixed input size cannot take both
- **Dependencies reach**: the word that decides the next may be six words back, or sixty — language is “an inherently temporal phenomenon”

A sequence model must take any length, respect the order, and let the distant past reach the present. No tool so far does all three.

Jurafsky & Martin (2026), Ch. 14 opening; Goodfellow et al. (2016), §10 opening.

What natural language processing asks of a model



Four architectures for NLP tasks, all with a sequence in: (a) sequence labelling maps each input token to an output token; (b) sequence classification maps the whole input to one class; (c) language modelling outputs the next token conditioned on the previous ones; (d) the encoder-decoder maps the input to a context and the context to an output sequence of its own length.

Jurafsky & Martin (2026), Fig. 14.15, §14.6.

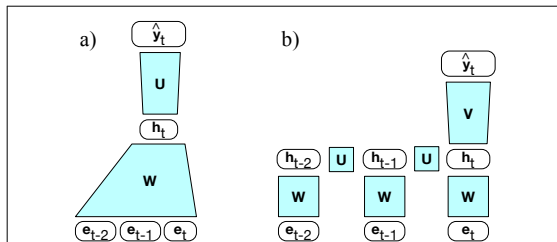
NLP, in one paragraph

- **Natural language processing**: getting a machine to read, understand and produce human language — text, and speech turned into text
- Its raw material is a sequence of discrete symbols — words, characters, sub-word pieces — each turned into a vector, an **embedding**: a one-hot input times a matrix
- Its fundamental move is to **predict**: the next word, the tag of this word, the sentiment of this review, the translation of this sentence
- Its difficulty is **context**: a word's meaning is fixed by words anywhere in the sentence — before it, or after it

The recurrent network is the first architecture built to read a sequence of any length, one token at a time, keeping what it has read.

Jurafsky & Martin (2026), Ch. 14 opening, §14.2, §14.3.

Why the tools we have fall short

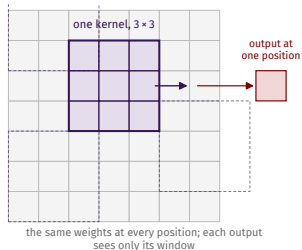


Two language-model architectures moving through a text, with a context of three tokens: (a) a feedforward model with a fixed context window into the weight matrix W ; (b) a recurrent model, in which the hidden state h_{t-1} summarises the prior context — all of it, back to the start. In Jurafsky & Martin's sentence *The flights the airline was canceling were full*, the verb *were* agrees with *flights* six words back: no fixed window reaches it, and the gap has no bound.

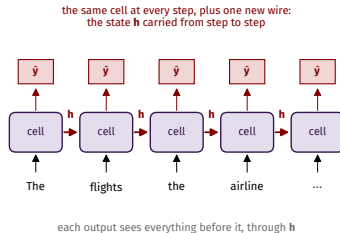
Jurafsky & Martin (2026), Fig. 14.5, §14.2, §14.5, example (14.19).

The idea: from sharing across space to sharing across time

(a) convolution: sharing across space



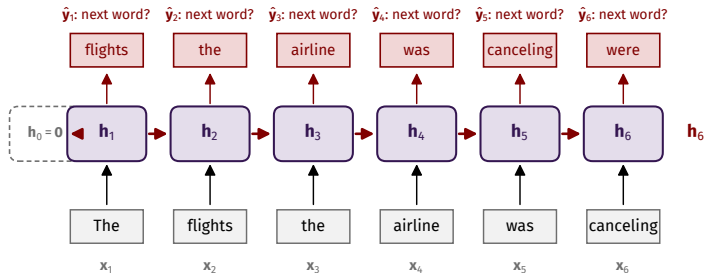
(b) recurrence: sharing across time



(a) A convolution applies the same kernel at every position of the grid; each output depends only on its window. (b) A recurrent network applies the same cell at every step of the sequence, and adds one wire: the state h , passed from each step to the next, so that each output depends on everything before it. The parallel is exact: sharing across space gave equivariance and few parameters; sharing across time gives a model of any length with few parameters, and the new wire buys memory.

Goodfellow et al. (2016), §10 opening, §10.1; Class 23.

How the recurrent network reads the sentence



“flights” enters the state at h_2 and is carried along the maroon wire;
it can still be in h_6 when the next word, “were”, must be predicted

The running example, read one word at a time. At each step the cell takes the current word x_t and the state h_{t-1} , produces a new state h_t — a summary of everything read so far — and from it a prediction $\hat{y}_t$ of the next word. *flights* enters the state at $t = 2$ and, if the network has learned to keep it, is still there at $t = 6$ when *were* is due. The state has a fixed size whatever the length of the sentence: a lossy summary, and training decides what it keeps.

Jurafsky & Martin (2026), §14.1–14.2; Goodfellow et al. (2016), §10.1.

The block ahead, and where today sits

Class	the problem	the answer
26 — today	how is a sequence read, and the reader trained?	the cell, unfolding, BPTT
27 — encoder–decoder	how is one sequence read and another written?	two chains, a context, attention
28 — LSTM and gates	why is the far past forgotten; what fixes it?	the gradient across time; gates
29 — NLP tasks	which output goes with which task?	language models, taggers, classifiers
30–31 — transformers	can a sequence be read without the chain?	attention without recurrence

- Today builds the machine and trains it; the classes after put it to work, open it up, and replace it. Two questions run through today: what does one step compute, and how does a loss at the end reach a weight used at the start?

Course plan, Classes 26–31; Jurafsky & Martin (2026), §14.6.

The Recurrent Network

A network with a cycle

- The picture two slides back has one feature no network of this course has had: an edge that goes **back** — the state **h** leaves the cell and re-enters it at the next step. Any network with such a cycle is a **recurrent neural network**
- Cycles in general are hard to reason about and train. Elman's **simple recurrent network** has exactly one: the hidden layer of step $t - 1$ feeds the hidden layer of step t
- That one edge is the memory: the previous hidden layer is a summary of the whole past — *flights* at $t = 6$ — with no fixed window
- The rest is ordinary: input, hidden and output layers, one feedforward step per token. So: what exactly does the step compute, and how does the cycle change training?

Jurafsky & Martin (2026), §14.1; Goodfellow et al. (2016), §10.1; Elman (1990).

Notation for today

$\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$	input at step t ; τ or n the sequence length; $\mathbf{h}_0 = \mathbf{0}$
$\mathbf{h}_t \in \mathbb{R}^{d_h}$	hidden state; $\mathbf{a}_t$ its pre-activation
$\mathbf{W}, \mathbf{U}, \mathbf{V}$	input-to-hidden, hidden-to-hidden, hidden-to-output; biases $\mathbf{b}, \mathbf{c}$
$\mathbf{o}_t, \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t)$	output scores and distribution; $\mathbf{y}_t$ the target, L_t the loss at t
g, g'	the hidden activation (tanh throughout) and its derivative, $1 - \mathbf{h}_t^2$
$\nabla_{\mathbf{h}_t} L$	the gradient of the total loss on the state at t
$\mathbf{h}_t^f, \mathbf{h}_t^b, [\cdot; \cdot]$	forward and backward states; concatenation; $L = \sum_t L_t$

Two conventions, one line

Jurafsky: **W** input, **U** recurrence, **V** output, time as a subscript. Goodfellow swaps **U** and **W** and writes $\mathbf{h}^{(t)}$. This block keeps Jurafsky's letters.

Jurafsky & Martin (2026), §14.1.1; Goodfellow et al. (2016), §10.2.

The simple recurrent network

Definition 1 (Simple recurrent network)

With $\mathbf{h}_0 = \mathbf{0}$, for $t = 1, \dots, \tau$:

$$\mathbf{a}_t = \mathbf{b} + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t, \quad \mathbf{h}_t = \tanh(\mathbf{a}_t), \quad \mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t, \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t).$$

The parameters $\mathbf{U}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}$ are the same at every step.

- Read it as the picture: the cell at step t takes two things, **what it sees now** ($\mathbf{x}_t$) and **what it remembers** ($\mathbf{h}_{t-1}$), mixes them, squashes, and hands the result on as the new memory $\mathbf{h}_t$
- From that memory it makes a prediction, $\hat{\mathbf{y}}_t$: a distribution over the next word, or the tag of this word — whatever the task asks for
- Forward inference is **incremental**: $\mathbf{h}_t$ needs $\mathbf{h}_{t-1}$, so the sequence is read start to end

Jurafsky & Martin (2026), Eqs. 14.1–14.3, Figs. 14.1–14.3; Goodfellow et al. (2016), Eqs. 10.8–10.11.

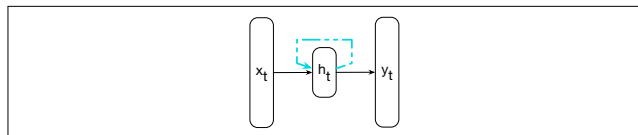
Each term, in words

term	what it is	in the running sentence, at <i>airline</i>
$\mathbf{x}_t$	the input at step t , an embedding	the vector for <i>airline</i>
$\mathbf{W}\mathbf{x}_t$	how the current input pushes the state	“a singular noun just came in”
$\mathbf{h}_{t-1}$	the memory: a summary of $\mathbf{x}_1 \dots \mathbf{x}_{t-1}$	“inside a clause about <i>flights</i> ”
$\mathbf{U}\mathbf{h}_{t-1}$	what of the memory is kept, and how it is rearranged	keep <i>flights</i> , note the clause
$\mathbf{b}, \tanh$	a bias, and a squash that keeps the state in $(-1, 1)$	nothing blows up over a long sentence
$\mathbf{h}_t$	the new memory, passed to step $t + 1$	everything so far, in d_h numbers
$\mathbf{V}\mathbf{h}_t$, softmax	the read-out: a score for every possible output	<i>was</i> likely, <i>were</i> less — for now

- Three matrices, three jobs: **W** reads, **U** remembers, **V** answers; the same three at every step, so their sizes do not depend on the sentence
- What **U** keeps and drops is learned from the loss — the second half of today

Jurafsky & Martin (2026), §14.1–14.1.1; Goodfellow et al. (2016), §10.1–10.2.

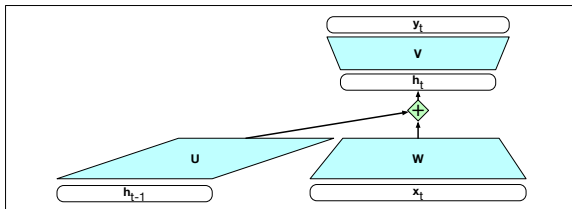
The simple recurrent network, drawn



The Elman network: the hidden layer includes a recurrent connection as part of its input (dashed), so its activation depends on the current input and on its own activation from the previous time step.

Jurafsky & Martin (2026), Fig. 14.1; Elman (1990).

The same network, as one feedforward step



The same network illustrated as a feedforward network: the hidden layer h_{t-1} from the prior time step is multiplied by U and added to the feedforward component Wx_t from the current time step; the only new set of weights is U .
Jurafsky & Martin (2026), Fig. 14.2.

Why unfold

- The picture with the cycle is compact but says nothing about **how to compute**: a cycle has no first node, and every gradient rule of Class 11 was for a graph without cycles
- The trick: give every time step its **own copy** of the cell. The cycle becomes a chain $\mathbf{h}_0 \rightarrow \mathbf{h}_1 \rightarrow \dots \rightarrow \mathbf{h}_\tau$, a directed acyclic graph as long as the sequence
- This buys three things: the forward pass is a walk along the chain; back-propagation is Class 11's algorithm on it, unchanged; and the parameters, shared by all the copies, are still one set, however long the chain
- It also puts today's two questions on the page: which computations happen, in what order, and along which paths a loss at the end reaches a weight used at the start

Goodfellow et al. (2016), §10.1; Jurafsky & Martin (2026), §14.1.2.

Unfolding, stated

Proposition 2 (Unfolding the recurrence)

The recurrence $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta)$, applied t times from $\mathbf{h}_0$, is a function of the whole past, $\mathbf{h}_t = g^{(t)}(\mathbf{x}_t, \dots, \mathbf{x}_1) = f(f(\dots f(\mathbf{h}_0, \mathbf{x}_1; \theta) \dots, \mathbf{x}_{t-1}; \theta), \mathbf{x}_t; \theta)$, a directed acyclic graph with t copies of f and one θ : the parameter count does not depend on t , the graph's size does.

- **Proof. Substitute the definition into itself $t - 1$ times; no recurrence remains.** □
- So the model's input size does not depend on the length, and one f with one θ serves every step: one model for all lengths, including unseen ones. The unfolded graph shows the paths of the forward and backward passes.

Goodfellow et al. (2016), §10.1, Eqs. 10.1–10.7, Figs. 10.1–10.2.

Unfolding, stated

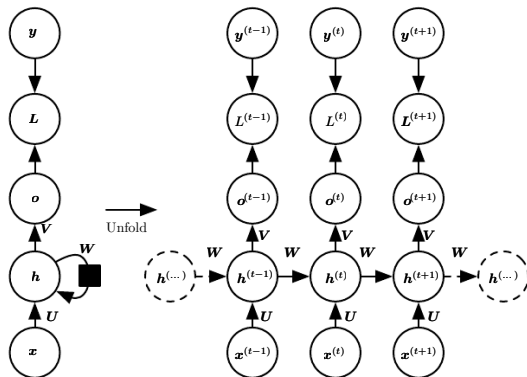
Proposition 2 (Unfolding the recurrence)

The recurrence $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t; \theta)$, applied t times from $\mathbf{h}_0$, is a function of the whole past, $\mathbf{h}_t = g^{(t)}(\mathbf{x}_t, \dots, \mathbf{x}_1) = f(f(\dots f(\mathbf{h}_0, \mathbf{x}_1; \theta) \dots, \mathbf{x}_{t-1}; \theta), \mathbf{x}_t; \theta)$, a directed acyclic graph with t copies of f and one θ : the parameter count does not depend on t , the graph's size does.

- Proof. Substitute the definition into itself $t - 1$ times; no recurrence remains. □
- So the model's input size does not depend on the length, and one f with one θ serves every step: one model for all lengths, including unseen ones. The unfolded graph shows the paths of the forward and backward passes.

Goodfellow et al. (2016), §10.1, Eqs. 10.1–10.7, Figs. 10.1–10.2.

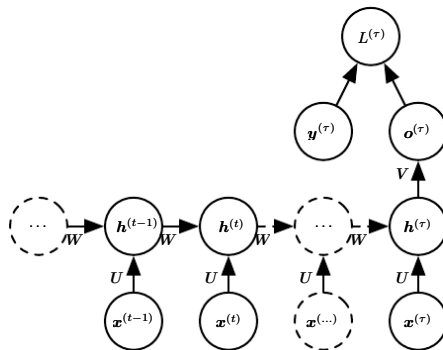
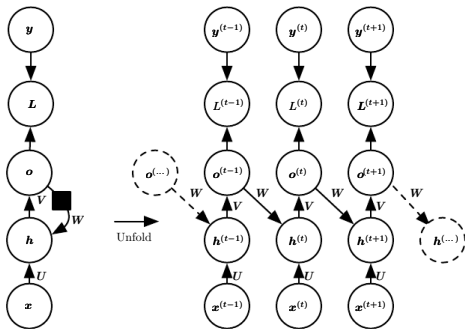
Unfolding, drawn



The recurrent network with outputs and losses, as a circuit (the black square is a delay of one step) and unfolded; its input-to-hidden, hidden-to-hidden and hidden-to-output matrices are Goodfellow's U , W , V — Jurafsky's W , U , V .

Goodfellow et al. (2016), Figs. 10.2–10.3.

Three design patterns



Beside the hidden-to-hidden network of Fig. 10.3: left, recurrence only from the output to the next hidden layer — less powerful, since o_{t-1} is all step t learns about the past, but trainable step by step (Class 27, teacher forcing); right, hidden recurrence with a single output after the whole sequence. Goodfellow et al. (2016), Figs. 10.4–10.5.

The loss of a sequence

- What is the network wrong about? In the running example it guessed the next word at every step: *flights* after *The*, ..., *were* after *canceling*. Each guess is a distribution $\hat{y}_t$ with a right answer y_t
- So there is a loss at **every step**, the cross-entropy of Class 11 — how little probability the guess gave the truth — and the loss of the sentence is their **sum**:

$$L(\{\mathbf{x}_1, \dots, \mathbf{x}_\tau\}, \{y_1, \dots, y_\tau\}) = \sum_{t=1}^{\tau} L_t = - \sum_{t=1}^{\tau} \log p_{\text{model}}(y_t \mid \mathbf{x}_1, \dots, \mathbf{x}_t),$$

with $p_{\text{model}}(y_t \mid \cdot)$ the entry of $\hat{y}_t$ for the true y_t

- Three sets of weights, **W**, **U**, **V**, each used at every step — so a mistake at step 6 is partly the fault of what **U** did at steps 2 to 5

Goodfellow et al. (2016), Eqs. 10.12–10.14; Jurafsky & Martin (2026), §14.1.2.

Back-propagation Through Time

Why two gradients

- The mistake at step 6: the network said *was*, the answer was *were*. Which weights should change, by how much? Three matrices, each used six times
- Class 11's answer for a feedforward network: walk the graph backwards, get the gradient on every **node** from the nodes after it, then read the **weights'** gradients off the edges. Same here, with two twists
- **Twist 1, the nodes.** h_3 has two descendants: its own output $\hat{y}_3$ and the next state h_4 . The blame reaching it has two parts — “your guess at step 3 was off” and “what you handed to step 4 made *its* guesses off” — so the gradient on the states is a **recursion** adding the two, right to left
- **Twist 2, the weights.** U is not one edge but six; its gradient is the **sum** of the blame on each use — the state gradient at that step, times what U was multiplying then
- Hence two theorems, states then parameters: **back-propagation through time**

Goodfellow et al. (2016), §10.2.2; Jurafsky & Martin (2026), §14.1.2; Class 11.

The gradient on the states

Theorem 3 (Back-propagation through time, I)

For the network of Definition 1 with $L = \sum_t L_t$ and $L_t = -\log \hat{y}_t[y_t]$:

$$\nabla_{\mathbf{o}_t} L = \hat{\mathbf{y}}_t - \mathbf{1}_{y_t}, \quad \nabla_{\mathbf{h}_\tau} L = \mathbf{V}^\top \nabla_{\mathbf{o}_\tau} L, \quad \nabla_{\mathbf{h}_t} L = \mathbf{U}^\top \text{diag}(1 - \mathbf{h}_{t+1}^2) \nabla_{\mathbf{h}_{t+1}} L + \mathbf{V}^\top \nabla_{\mathbf{o}_t} L \quad (t < \tau).$$

- **Proof.** $\partial L / \partial L_t = 1$, and the softmax cross-entropy has gradient $\hat{\mathbf{y}}_t - \mathbf{1}_{y_t}$ on its scores (Class 11). At the last step $\mathbf{h}_\tau$ has one descendant, $\mathbf{o}_\tau = \mathbf{c} + \mathbf{V}\mathbf{h}_\tau$. For $t < \tau$ it has two, $\mathbf{o}_t$ and $\mathbf{h}_{t+1}$; the chain rule adds a term for each, and $\partial \mathbf{h}_{t+1} / \partial \mathbf{h}_t = \text{diag}(1 - \mathbf{h}_{t+1}^2) \mathbf{U}$. □
- A **recursion** that walks the unfolded graph right to left, as the forward pass walked it left to right: Class 11's back-propagation, on the unfolded graph.

Goodfellow et al. (2016), §10.2.2, Eqs. 10.17–10.21; Jurafsky & Martin (2026), §14.1.2; Werbos (1990).

The gradient on the states

Theorem 3 (Back-propagation through time, I)

For the network of Definition 1 with $L = \sum_t L_t$ and $L_t = -\log \hat{y}_t[y_t]$:

$$\nabla_{\mathbf{o}_t} L = \hat{\mathbf{y}}_t - \mathbf{1}_{y_t}, \quad \nabla_{\mathbf{h}_\tau} L = \mathbf{V}^\top \nabla_{\mathbf{o}_\tau} L, \quad \nabla_{\mathbf{h}_t} L = \mathbf{U}^\top \text{diag}(1 - \mathbf{h}_{t+1}^2) \nabla_{\mathbf{h}_{t+1}} L + \mathbf{V}^\top \nabla_{\mathbf{o}_t} L \quad (t < \tau).$$

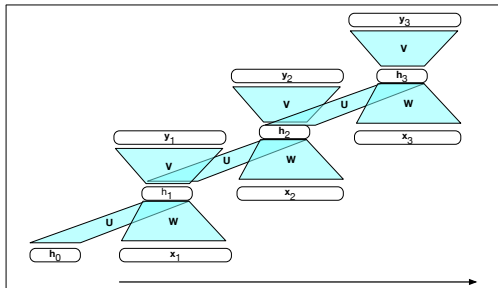
- Proof. $\partial L / \partial L_t = 1$, and the softmax cross-entropy has gradient $\hat{\mathbf{y}}_t - \mathbf{1}_{y_t}$ on its scores (Class 11). At the last step $\mathbf{h}_\tau$ has one descendant, $\mathbf{o}_\tau = \mathbf{c} + \mathbf{V}\mathbf{h}_\tau$. For $t < \tau$ it has two, $\mathbf{o}_t$ and $\mathbf{h}_{t+1}$; the chain rule adds a term for each, and $\partial \mathbf{h}_{t+1} / \partial \mathbf{h}_t = \text{diag}(1 - \mathbf{h}_{t+1}^2) \mathbf{U}$. □
- **A recursion that walks the unfolded graph right to left, as the forward pass walked it left to right: Class 11's back-propagation, on the unfolded graph.**

Goodfellow et al. (2016), §10.2.2, Eqs. 10.17–10.21; Jurafsky & Martin (2026), §14.1.2; Werbos (1990).

Forward inference, and the unrolled network

```
function FORWARDRNN(x, network) returns output sequence y
  h0 ← 0
  for i ← 1 to LENGTH(x) do
    hi ← g(Uhi-1 + Wxi)
    yi ← f(Vhi)
  return y
```

Forward inference: U, V, W shared across time, new h and y at each step.
Jurafsky & Martin (2026), Fig. 14.3.



The network unrolled in time: the layers are recalculated at each step, the weights shared across all steps. The backward pass walks it right to left, computing the error term at each h_t from its own output and from h_{t+1} . Jurafsky & Martin (2026), Fig. 14.4.

Jurafsky & Martin (2026), §14.1.1–14.1.2.

The gradient on the parameters

Theorem 4 (Back-propagation through time, II)

The gradients on the shared parameters are sums over time:

$$\nabla_{\mathbf{V}}L = \sum_t (\nabla_{\mathbf{o}_t}L) \mathbf{h}_t^\top, \quad \nabla_{\mathbf{U}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2)(\nabla_{\mathbf{h}_t}L) \mathbf{h}_{t-1}^\top, \quad \nabla_{\mathbf{W}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2)(\nabla_{\mathbf{h}_t}L) \mathbf{x}_t^\top,$$

$$\nabla_{\mathbf{c}}L = \sum_t \nabla_{\mathbf{o}_t}L, \quad \nabla_{\mathbf{b}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2) \nabla_{\mathbf{h}_t}L.$$

- **Proof.** Write $\mathbf{U}_{(t)}$ for the copy of $\mathbf{U}$ used at step t ; a parameter on many edges contributes through each, $\nabla_{\mathbf{U}}L = \sum_t \nabla_{\mathbf{U}_{(t)}}L$. $\mathbf{a}_t$ is linear in each copy, so the copy's gradient is the outer product of $\nabla_{\mathbf{a}_t}L = \text{diag}(1 - \mathbf{h}_t^2)\nabla_{\mathbf{h}_t}L$ with its input. □
- The copies resolve an ambiguity: $\nabla_{\mathbf{U}}$ in calculus counts every edge, one node's back-propagation counts one. No gradient on $\mathbf{x}_t$ is needed.

Goodfellow et al. (2016), §10.2.2, Eqs. 10.22–10.28.

The gradient on the parameters

Theorem 4 (Back-propagation through time, II)

The gradients on the shared parameters are sums over time:

$$\nabla_{\mathbf{V}}L = \sum_t (\nabla_{\mathbf{o}_t}L) \mathbf{h}_t^\top, \quad \nabla_{\mathbf{U}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2)(\nabla_{\mathbf{h}_t}L) \mathbf{h}_{t-1}^\top, \quad \nabla_{\mathbf{W}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2)(\nabla_{\mathbf{h}_t}L) \mathbf{x}_t^\top,$$

$$\nabla_{\mathbf{c}}L = \sum_t \nabla_{\mathbf{o}_t}L, \quad \nabla_{\mathbf{b}}L = \sum_t \text{diag}(1 - \mathbf{h}_t^2) \nabla_{\mathbf{h}_t}L.$$

- Proof. Write $\mathbf{U}_{(t)}$ for the copy of $\mathbf{U}$ used at step t ; a parameter on many edges contributes through each, $\nabla_{\mathbf{U}}L = \sum_t \nabla_{\mathbf{U}_{(t)}}L$. $\mathbf{a}_t$ is linear in each copy, so the copy's gradient is the outer product of $\nabla_{\mathbf{a}_t}L = \text{diag}(1 - \mathbf{h}_t^2)\nabla_{\mathbf{h}_t}L$ with its input. □
- **The copies resolve an ambiguity: $\nabla_{\mathbf{U}}$ in calculus counts every edge, one node's back-propagation counts one. No gradient on $\mathbf{x}_t$ is needed.**

Goodfellow et al. (2016), §10.2.2, Eqs. 10.22–10.28.

Where the gradient comes from

Corollary 5 (The gradient across a lag is a product)

Iterating the recursion of Theorem 3 for a loss at the last step only,

$$\nabla_{\mathbf{h}_k} L_\tau = \left(\prod_{s=k+1}^{\tau} \mathbf{U}^\top \text{diag}(1 - \mathbf{h}_s^2) \right) \mathbf{v}^\top \nabla_{\mathbf{o}_\tau} L_\tau :$$

the same matrix $\mathbf{U}^\top$, times an activation derivative, multiplied in once per step of lag.

- **Proof.** With $\nabla_{\mathbf{o}_t} L_\tau = 0$ for $t < \tau$ the second term of the recursion vanishes; unroll it. □
- The same product feeds every per-step term of Theorem 4: what reaches $\mathbf{U}$ from step k is this, times $\mathbf{h}_{k-1}^\top$. So the terms of the sum over time are not equal, and a repeated multiplication by one matrix is the power iteration of Class 13 — **Class 28 starts here.**

Goodfellow et al. (2016), Eq. 10.21 iterated, §10.7; Class 13; Class 28.

Where the gradient comes from

Corollary 5 (The gradient across a lag is a product)

Iterating the recursion of Theorem 3 for a loss at the last step only,

$$\nabla_{\mathbf{h}_k} L_\tau = \left(\prod_{s=k+1}^{\tau} \mathbf{U}^\top \text{diag}(1 - \mathbf{h}_s^2) \right) \mathbf{v}^\top \nabla_{\mathbf{o}_\tau} L_\tau :$$

the same matrix $\mathbf{U}^\top$, times an activation derivative, multiplied in once per step of lag.

- Proof. With $\nabla_{\mathbf{o}_t} L_\tau = \mathbf{0}$ for $t < \tau$ the second term of the recursion vanishes; unroll it. □
- The same product feeds every per-step term of Theorem 4: what reaches $\mathbf{U}$ from step k is this, times $\mathbf{h}_{k-1}^\top$. So the terms of the sum over time are not equal, and a repeated multiplication by one matrix is the power iteration of Class 13 — Class 28 starts here.

Goodfellow et al. (2016), Eq. 10.21 iterated, §10.7; Class 13; Class 28.

The Cost of Time

Two passes, sequential

Proposition 6 (Cost of the unrolled graph)

For a sequence of length τ , the forward pass and the backward pass each take $O(\tau)$ time, the forward pass cannot be parallelised across steps, and the backward pass needs every $\mathbf{h}_t$ from the forward pass, so the memory is $O(\tau)$ as well.

- **Proof.** One step of Definition 1 costs a fixed amount and needs $\mathbf{h}_{t-1}$, so τ steps cost τ times that, in order. Theorem 4 multiplies by $\mathbf{h}_t$, $\mathbf{h}_{t-1}$ and $1 - \mathbf{h}_t^2$ at every t , so all of them must be kept. □
- Contrast convolution, whose positions run at once. Depth in time is paid serially — one reason the transformer (Classes 30–31) gives the recurrence up.
- No specialised algorithm is needed: unroll the network for the given input, and ordinary back-propagation on that graph is BPTT. Class 27's tape does exactly this.

Goodfellow et al. (2016), §10.2; Jurafsky & Martin (2026), §14.1.2.

Two passes, sequential

Proposition 6 (Cost of the unrolled graph)

For a sequence of length τ , the forward pass and the backward pass each take $O(\tau)$ time, the forward pass cannot be parallelised across steps, and the backward pass needs every $\mathbf{h}_t$ from the forward pass, so the memory is $O(\tau)$ as well.

- Proof. One step of Definition 1 costs a fixed amount and needs $\mathbf{h}_{t-1}$, so τ steps cost τ times that, in order. Theorem 4 multiplies by $\mathbf{h}_t$, $\mathbf{h}_{t-1}$ and $1 - \mathbf{h}_t^2$ at every t , so all of them must be kept. □
- **Contrast convolution, whose positions run at once. Depth in time is paid serially — one reason the transformer (Classes 30–31) gives the recurrence up.**
- No specialised algorithm is needed: unroll the network for the given input, and ordinary back-propagation on that graph is BPTT. Class 27's tape does exactly this.

Goodfellow et al. (2016), §10.2; Jurafsky & Martin (2026), §14.1.2.

Two passes, sequential

Proposition 6 (Cost of the unrolled graph)

For a sequence of length τ , the forward pass and the backward pass each take $O(\tau)$ time, the forward pass cannot be parallelised across steps, and the backward pass needs every $\mathbf{h}_t$ from the forward pass, so the memory is $O(\tau)$ as well.

- Proof. One step of Definition 1 costs a fixed amount and needs $\mathbf{h}_{t-1}$, so τ steps cost τ times that, in order. Theorem 4 multiplies by $\mathbf{h}_t$, $\mathbf{h}_{t-1}$ and $1 - \mathbf{h}_t^2$ at every t , so all of them must be kept. □
- Contrast convolution, whose positions run at once. Depth in time is paid serially — one reason the transformer (Classes 30–31) gives the recurrence up.
- **No specialised algorithm is needed: unroll the network for the given input, and ordinary back-propagation on that graph is BPTT. Class 27's tape does exactly this.**

Goodfellow et al. (2016), §10.2; Jurafsky & Martin (2026), §14.1.2.

Truncation

Proposition 7 (What truncated BPTT removes)

Cut the backward pass after k steps: treat $\mathbf{h}_{\tau-k}$ as a constant. Then every term of Theorem 4 with $t \leq \tau - k$ is dropped, and a loss at τ sends no gradient to any parameter use, or any input, more than k steps before it.

- **Proof.** With $\mathbf{h}_{\tau-k}$ constant the recursion stops there: $\nabla_{\mathbf{h}_t} L = 0$ for $t \leq \tau - k$, and every per-step term with that factor is zero. The forward pass is unchanged. □
- Why: a long input — speech, characters, a stream — may not unroll in full, so it is cut into fixed-length segments, each a training item. The state still crosses the cut forward; the gradient does not.
- The price: a dependency longer than k can be used by the forward pass but never **learned** by the backward one.

Jurafsky & Martin (2026), §14.1.2, in words; Williams & Peng (1990); Class 27.

Truncation

Proposition 7 (What truncated BPTT removes)

Cut the backward pass after k steps: treat $\mathbf{h}_{\tau-k}$ as a constant. Then every term of Theorem 4 with $t \leq \tau - k$ is dropped, and a loss at τ sends no gradient to any parameter use, or any input, more than k steps before it.

- Proof. With $\mathbf{h}_{\tau-k}$ constant the recursion stops there: $\nabla_{\mathbf{h}_t} L = \mathbf{0}$ for $t \leq \tau - k$, and every per-step term with that factor is zero. The forward pass is unchanged. □
- **Why: a long input — speech, characters, a stream — may not unroll in full, so it is cut into fixed-length segments, each a training item. The state still crosses the cut forward; the gradient does not.**
- The price: a dependency longer than k can be used by the forward pass but never **learned** by the backward one.

Jurafsky & Martin (2026), §14.1.2, in words; Williams & Peng (1990); Class 27.

Truncation

Proposition 7 (What truncated BPTT removes)

Cut the backward pass after k steps: treat $\mathbf{h}_{\tau-k}$ as a constant. Then every term of Theorem 4 with $t \leq \tau - k$ is dropped, and a loss at τ sends no gradient to any parameter use, or any input, more than k steps before it.

- Proof. With $\mathbf{h}_{\tau-k}$ constant the recursion stops there: $\nabla_{\mathbf{h}_t} L = \mathbf{0}$ for $t \leq \tau - k$, and every per-step term with that factor is zero. The forward pass is unchanged. □
- Why: a long input — speech, characters, a stream — may not unroll in full, so it is cut into fixed-length segments, each a training item. The state still crosses the cut forward; the gradient does not.
- **The price: a dependency longer than k can be used by the forward pass but never learned by the backward one.**

Jurafsky & Martin (2026), §14.1.2, in words; Williams & Peng (1990); Class 27.

Both Directions

Why a second direction

- Everything so far reads **left to right**: $\mathbf{h}_t$ summarises $\mathbf{x}_1 \dots \mathbf{x}_t$ and knows nothing after. For predicting the next word that is right: the future is what is being predicted
- But many tasks have the **whole input in hand**, and the answer at t depends on what follows. Tag *back* in “*back* the bill” (verb) against “*back* and neck ached” (noun): at $t = 1$ a left-to-right reader has seen only *back*
- Speech likewise: which phoneme a sound is depends on the next few, by co-articulation; which word, on the next few words
- The fix is not a bigger window but a **second chain**, read from the end, whose state at t summarises $\mathbf{x}_t \dots \mathbf{x}_n$; side by side, the two states let the output at t see both sides

Goodfellow et al. (2016), §10.3; Jurafsky & Martin (2026), §14.4.2; Class 29.

Reading the sequence both ways

Definition 8 (Bidirectional RNN)

Two independent recurrent networks, one read from the start and one from the end, their states concatenated at each step:

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \dots, \mathbf{x}_t), \quad \mathbf{h}_t^b = \text{RNN}_{\text{backward}}(\mathbf{x}_t, \dots, \mathbf{x}_n), \quad \mathbf{h}_t = [\mathbf{h}_t^f; \mathbf{h}_t^b],$$

and the output $\mathbf{o}_t$ computed from $\mathbf{h}_t$.

- Every network so far is **causal**: $\mathbf{h}_t$ knows nothing after $\mathbf{x}_t$. Many predictions need the future too — a phoneme depends on the next few, a word on the next words
- $\mathbf{h}_t^b$ is the ordinary state of a network run on the reversed input; Goodfellow's $\mathbf{g}^{(t)}$ is Jurafsky's $\mathbf{h}_t^b$. Sum or product also combine them; four chains do the same on a 2-D grid

Jurafsky & Martin (2026), §14.4.2, Eqs. 14.16–14.18; Goodfellow et al. (2016), §10.3; Schuster & Paliwal (1997).

What the second direction changes

Proposition 9 (The bidirectional output sees the whole input)

In a bidirectional RNN the output $\mathbf{o}_t$ is a function of $\mathbf{x}_1, \dots, \mathbf{x}_n$ for every t , most sensitive to the inputs near t but with no fixed window; the cost is twice one direction; and the network is not causal, so it needs the whole input before any output and cannot generate a sequence step by step.

- **Proof.** h_t^f depends on $\mathbf{x}_{\leq t}$ and h_t^b on $\mathbf{x}_{\geq t}$ by Proposition 2 applied to each chain; $\mathbf{o}_t$ reads both. Each chain is one forward pass of Proposition 6. □
- Training is unchanged: unroll both chains and back-propagate; the backward chain's BPTT runs left to right.
- Used for labelling and classification (Class 29) and the encoder of Class 27; not for language modelling or decoding, which emit the next token from the past alone.

Goodfellow et al. (2016), §10.3; Jurafsky & Martin (2026), §14.4.2.

What the second direction changes

Proposition 9 (The bidirectional output sees the whole input)

In a bidirectional RNN the output $\mathbf{o}_t$ is a function of $\mathbf{x}_1, \dots, \mathbf{x}_n$ for every t , most sensitive to the inputs near t but with no fixed window; the cost is twice one direction; and the network is not causal, so it needs the whole input before any output and cannot generate a sequence step by step.

- Proof. $\mathbf{h}_t^f$ depends on $\mathbf{x}_{\leq t}$ and $\mathbf{h}_t^b$ on $\mathbf{x}_{\geq t}$ by Proposition 2 applied to each chain; $\mathbf{o}_t$ reads both. Each chain is one forward pass of Proposition 6. □
- **Training is unchanged: unroll both chains and back-propagate; the backward chain's BPTT runs left to right.**
- Used for labelling and classification (Class 29) and the encoder of Class 27; not for language modelling or decoding, which emit the next token from the past alone.

Goodfellow et al. (2016), §10.3; Jurafsky & Martin (2026), §14.4.2.

What the second direction changes

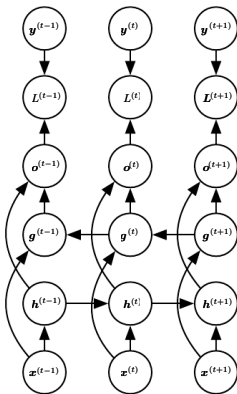
Proposition 9 (The bidirectional output sees the whole input)

In a bidirectional RNN the output $\mathbf{o}_t$ is a function of $\mathbf{x}_1, \dots, \mathbf{x}_n$ for every t , most sensitive to the inputs near t but with no fixed window; the cost is twice one direction; and the network is not causal, so it needs the whole input before any output and cannot generate a sequence step by step.

- Proof. $\mathbf{h}_t^f$ depends on $\mathbf{x}_{\leq t}$ and $\mathbf{h}_t^b$ on $\mathbf{x}_{\geq t}$ by Proposition 2 applied to each chain; $\mathbf{o}_t$ reads both. Each chain is one forward pass of Proposition 6. □
- Training is unchanged: unroll both chains and back-propagate; the backward chain's BPTT runs left to right.
- **Used for labelling and classification (Class 29) and the encoder of Class 27; not for language modelling or decoding, which emit the next token from the past alone.**

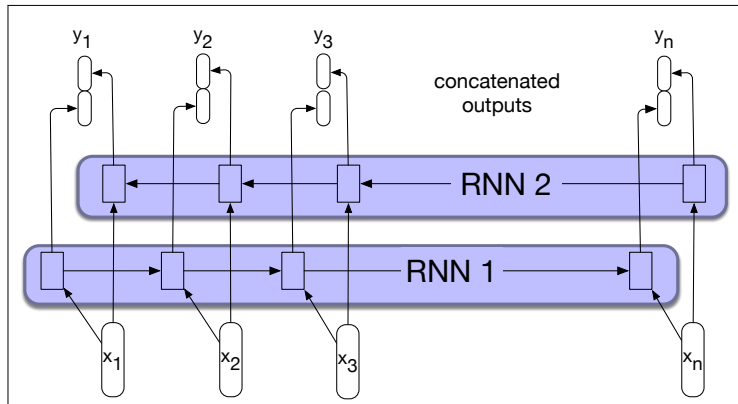
Goodfellow et al. (2016), §10.3; Jurafsky & Martin (2026), §14.4.2.

Both directions, drawn



A bidirectional RNN: h is the state of the sub-RNN that moves forward through time, g the state of the one that moves backward; the output units $o^{(t)}$ compute a representation that depends on both the past and the future, most sensitive to the inputs near t . Goodfellow et al. (2016), Fig. 10.11, §10.3.

Both directions, as two networks



Separate models are trained in the forward and backward directions, and the output of each network at each time point is concatenated to represent the bidirectional state at that time point.

Jurafsky & Martin (2026), Fig. 14.11, §14.4.2.

Putting It Together

The block ahead, revisited

Class	takes from today	adds
27 — encoder–decoder	the cell as a black box; unfolding; BPTT	two chains, a context, attention
28 — LSTM and gates	Corollary 5: the product of Jacobians	why it vanishes, a path that does not
29 — NLP tasks	Definition 8; the three design patterns	what to read out, where to pay the loss
30–31 — transformers	Proposition 6: time is paid serially	attention without recurrence

Unfold, and a recurrent network is a deep feedforward network with its weights tied across layers. Everything about training it follows from that one picture.

Takeaways

1. A sequence has any length, an order, and dependencies that reach; a window cannot follow them, a state carried step by step can — the same weight sharing as convolution, across time instead of space
2. A simple recurrent network is one feedforward step with two inputs, the current $\mathbf{x}_t$ and the previous $\mathbf{h}_{t-1}$, applied with the same $\mathbf{U}, \mathbf{W}, \mathbf{V}$ at every step (Definition 1); unfolded, it is a DAG whose size is the sequence length and whose parameter count is not (Proposition 2)
3. Back-propagation through time is back-propagation on that graph: a backward recursion over the states, then sums over time for the shared parameters (Theorems 3–4)
4. Across a lag the gradient is a product of one matrix and one derivative per step (Corollary 5), so the recent steps dominate the sum
5. Both passes are $O(\tau)$ in time and memory and serial in time (Proposition 6); truncating the backward pass to k steps removes every path longer than k (Proposition 7)
6. A bidirectional network runs a second chain from the end; its output at t sees the whole input and it is no longer causal (Definition 8, Proposition 9)

Class 27: two of these networks, one reading and one writing — the encoder–decoder.

References i

Questions?